DESENVOLVIMENTO MOBILE · MÓDULO 18 · COMUM ÀS TRILHAS · NÍVEL AVANÇADO

Engenharia Avançada

Dívida técnica, modularização, desempenho, memória, observabilidade, SLOs, CI/CD com GitHub Actions, *feature flags* e decisões de arquitetura

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Reconhecer e gerenciar dívida técnica.
- Dividir um app em módulos com dependências bem definidas, em Gradle e em pacotes Dart.
- Explicar o orçamento de quadro, medir desempenho e corrigir vazamentos de memória.
- Instrumentar o app com *logs*, métricas e *traces* e definir SLOs.
- Montar *pipelines* de CI/CD para Android e Flutter.
- Lançar com segurança usando *feature flags* e registrar decisões de arquitetura.

Roteiro

Neste módulo

- Fundamentos: software que cresce
- Modularização
- Desempenho
- Observabilidade
- CI/CD
- Decisões e trabalho em equipe
- Fechamento

Fundamentos: software que cresce

O que muda quando o app cresce

Cresce	Problema que aparece
Código	*build* lento, mudanças com efeitos inesperados
Equipe	conflitos, decisões esquecidas, revisão lenta
Usuários	aparelhos fracos, redes ruins, falhas raras que viram milhares
Entregas	lançamentos manuais arriscados e demorados

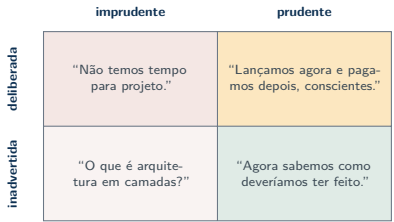
Um pouco de história

Nos anos 1970, Meir Lehman observou "leis" da evolução de software: um sistema em uso precisa mudar continuamente, e sua complexidade cresce a menos que se trabalhe ativamente para reduzi-la.

As respostas deste módulo

- **Modularização** para o código e a equipe.
- **Medição de desempenho** para os aparelhos reais.
- **Observabilidade** para enxergar o app em produção.
- **CI/CD** e *feature flags* para entregar com frequência e segurança.
- **Registros de decisão** para a memória da equipe.

Dívida técnica



Quadrante de Martin Fowler.

Dívida técnica

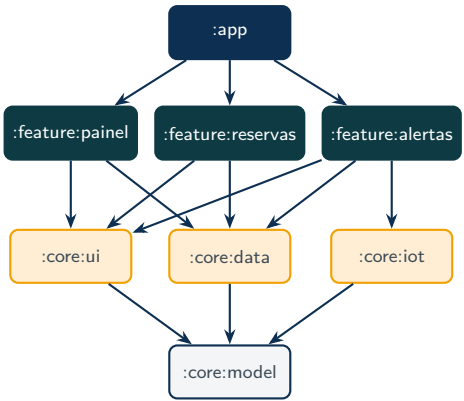
Metáfora de Ward Cunningham (1992): atalhos no código são como um empréstimo. Entregam mais rápido hoje, mas cobram **juros** em cada mudança futura, até serem "pagos" com refatoração.

Gerenciando

- Registre a dívida como *issue*, com o custo que ela causa.
- Reserve parte de cada *sprint* para pagá-la.
- Pague primeiro onde o código muda com mais frequência.

Modularização

Por que dividir o app em módulos



Benefícios

- *Build* incremental e em paralelo: só recompila o que mudou.
- Encapsulamento: o que é interno não vaza (`internal`, `src/`).
- Times trabalham em módulos diferentes sem conflito.
- Módulos de base reutilizáveis (outro app, servidor, painel web).

Regras do grafo

Dependências só para baixo; *features* não dependem umas das outras; sem ciclos.

Módulos nas duas trilhas

Kotlin: settings.gradle.kts

```
include(
  ":app",
  ":core:model", ":core:data", ":core:ui",
  ":feature:painel", ":feature:reservas",
)
```

feature/painel/build.gradle.kts

```
plugins {
  id("labmonitor.android.feature")  // convenção
}
dependencies {
  implementation(project(":core:data"))
  implementation(project(":core:ui"))
}
```

Flutter: pubspec.yaml da raiz

```
name: labmonitor_workspace
environment:
  sdk: ^3.6.0
workspace:
  - app
  - packages/core_model
  - packages/core_data
  - packages/feature_painel
```

packages/feature_painel/pubspec.yaml

```
name: feature_painel
resolution: workspace
dependencies:
  core_data: any
  flutter: { sdk: flutter }
```

Dica

Pub workspaces exigem Dart 3.6 ou mais novo.

Desempenho

A física do desempenho

Orçamento de quadro

Para animar com fluidez, cada quadro precisa ficar pronto antes do próximo *refresh* da tela: 16,7 ms a 60 Hz, 11,1 ms a 90 Hz e 8,3 ms a 120 Hz. Quadro atrasado = *jank* (engasgo).



Inicialização	Situação
A frio	processo não existia: tudo é criado
Morna	processo existe, tela recriada
A quente	app volta do fundo

Limites do Android vitals

São considerados excessivos: 5 s ou mais a frio, 2 s a morna e 1,5 s a quente. Sem resposta a um toque por 5 s, o Android mostra um ANR.

Dica

Rádio de celular gasta energia para "acordar": agrupe requisições em vez de fazer muitas pequenas.

Medir antes de otimizar

O método

1. Reproduza num **aparelho real** e modesto.
2. Meça em modo *release* ou *profile*, nunca em *debug*.
3. Encontre o gargalo com o *profiler*.
4. Mude **uma** coisa e meça de novo.
5. Proteja o ganho com um teste de desempenho.

Um pouco de história

Donald Knuth (1974): a otimização prematura é a raiz de todos os males.

Ferramenta	Para quê
Android Studio Profiler	CPU, memória, rede e energia (Kotlin)
Perfetto	rastreamento do sistema inteiro, quadro a quadro
Macrobenchmark	medir inicialização e rolagem de forma reprodutível
Baseline Profiles	pré-compilar o código mais usado e acelerar a inicialização
Flutter DevTools	quadros, CPU, memória, rede e reconstruções de widgets

Técnicas que mais rendem

	Compose (Kotlin)	Flutter
Evitar trabalho repetido	`remember`, `derivedStateOf`, parâmetros estáveis	construtores `const`, dividir widgets, `ValueListenableBuilder`
Listas longas	`LazyColumn` com `key`	`ListView.builder` com chaves
Trabalho pesado	`Dispatchers.Default`	`Isolate.run` (Módulo 06)
Imagens	tamanho certo (Coil), cache	`cacheWidth`, cache, formato adequado
Desenho	evitar recomposição na fase de desenho	`RepaintBoundary` em áreas animadas
Inicialização	*Baseline Profiles*, iniciar sob demanda	adiar inicializações, *deferred imports*

Comum às duas

Paginação, cache local, menos dados por requisição, nada de E/S na *thread* principal.

Dica

O Impeller pré-compila os *shaders* do Flutter, eliminando o engasgo da primeira animação que existia no renderizador antigo.

Memória e vazamentos

Vazamento de memória

Objeto inútil, mas ainda **alcançável** por alguma referência: o coletor de lixo não pode liberá-lo.

Causas típicas

Listeners e *streams* não cancelados, *controllers* não descartados, referência estática a uma tela.

Dica

LeakCanary (Android) e o DevTools detectam vazamentos.

Kotlin

```kotlin
// coleta só com a tela visível
lifecycleScope.launch {
  repeatOnLifecycle(
      Lifecycle.State.STARTED) {
    vm.estado.collect {
      desenhar(it)
    }
  }
}
// em Compose:
// collectAsStateWithLifecycle()
```

Flutter

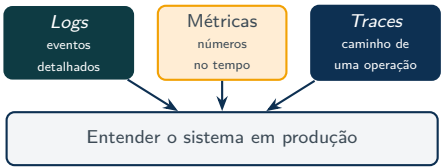
```dart
late final _ctrl =
    TextEditingController();
StreamSubscription? _sub;

@override
void dispose() {
  _ctrl.dispose();
  _sub?.cancel();
  super.dispose();
}
```

Quem cria, descarta: todo recurso aberto num ciclo de vida é fechado no mesmo ciclo.

Observabilidade

Os três pilares da observabilidade



Observabilidade

Capacidade de entender o que acontece dentro do sistema a partir dos dados que ele emite, inclusive para problemas que ninguém previu.

No app	Exemplo
Falhas	Crashlytics: pilha, aparelho, versão
Métricas	tempo de inicialização, quadros lentos
Traces	"reservar": toque → API → banco
Eventos	reservas por dia, telas mais usadas

Dica

OpenTelemetry é o padrão aberto para emitir os três tipos de dados, no app e no servidor.

Logs úteis e rastreamento ponta a ponta

Dart (pacote logging)

```dart
final log = Logger('reservas');
log.info('reserva criada', {
  'bancada': 7, 'duracao_min': 120,
  'requisicao': idRequisicao,
});
log.severe('falha ao reservar', erro, pilha);
```

Kotlin

```kotlin
Log.i("reservas",
  "reserva criada bancada=7 req=$idRequisicao")
```

Boas práticas

- Níveis: *debug*, *info*, *warning*, *error*.
- Campos estruturados, não frases soltas.
- Nunca senhas, *tokens* ou dados pessoais.
- Em *release*, só o necessário.

ID de correlação

O app gera um identificador por operação e o envia no cabeçalho `X-Request-ID`; app e servidor registram o mesmo ID, e a busca por ele mostra o caminho completo.

SLIs, SLOs e orçamento de erro

SLI — indicador de nível de serviço

Medida da experiência do usuário: % de sessões sem falha, % de reservas concluídas em menos de 2 s.

SLO — objetivo de nível de serviço

Meta para o SLI num período: "99,5 % das sessões sem falha em 28 dias".

Orçamento de erro

O que sobra até 100 %: 0,5 % das sessões. Enquanto há orçamento, lança-se rápido; se ele acaba, a prioridade vira estabilidade.

Conta rápida

Com 20 000 sessões no mês e SLO de 99,5 %, o orçamento é de 100 sessões com falha. Uma versão que gera 80 falhas na primeira semana já consumiu 80 % dele: hora de pausar novidades e corrigir.

Alertas

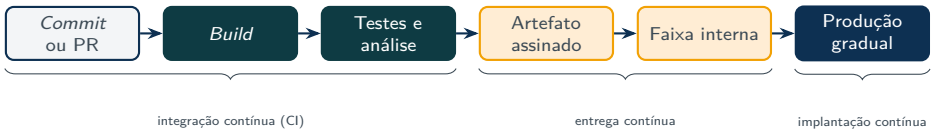
Alerte sobre **sintomas** que o usuário sente (falhas, lentidão), não sobre cada detalhe interno, e só quando alguém precisa agir.

Um pouco de história

SLOs e orçamentos de erro foram difundidos pela engenharia de confiabilidade (SRE) do Google.

CI/CD

Integração, entrega e implantação contínuas



CI

Integrar na *main* várias vezes ao dia, com *build* e testes automáticos a cada mudança.

Entrega contínua

Toda mudança aprovada gera uma versão pronta para publicar com um clique.

Implantação contínua

A versão vai sozinha para os usuários. Em apps, é sempre gradual e com monitoramento.

Pipeline Android (Kotlin)

.github/workflows/android.yml

```yaml
name: android
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 17 }
      - uses: gradle/actions/setup-gradle@v4
      - run: ./gradlew lint testDebugUnitTest
      - if: github.ref == 'refs/heads/main'
        run: |
          echo "$CHAVE_B64" | base64 -d > app/upload.jks
          ./gradlew bundleRelease
        env:
          CHAVE_B64: ${{ secrets.CHAVE_UPLOAD_B64 }}
          SENHA_CHAVE: ${{ secrets.SENHA_CHAVE }}
      - if: github.ref == 'refs/heads/main'
        uses: actions/upload-artifact@v4
        with: { name: aab, path: app/build/outputs/bundle/release/ }
```

Leitura

Em todo PR: *lint* e testes. Na *main*: também o AAB assinado, com a chave vinda dos segredos do repositório.

Dica

O passo seguinte envia o AAB à faixa interna da Play (*action* de upload ou fastlane `supply`, Módulo 15).

Atenção

Segredos nunca aparecem no *log*: o GitHub os mascara, mas não os imprima de propósito.

Pipeline Flutter

.github/workflows/flutter.yml

```yaml
name: flutter
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  qualidade:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: subosito/flutter-action@v2
        with: { channel: stable, cache: true }
      - run: flutter pub get
      - run: dart format --set-exit-if-changed .
      - run: flutter analyze
      - run: flutter test --coverage
  ios:
    needs: qualidade
    if: github.ref == 'refs/heads/main'
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v5
      - uses: subosito/flutter-action@v2
      - run: cd ios && fastlane beta    # Módulo 15
```

Dependência entre *jobs*

`needs` faz o *job* iOS esperar a qualidade passar; ele roda numa máquina macOS, exigida para compilar para iOS.

Dica

Use *environments* do GitHub com aprovação manual para o passo que publica em produção.

Atenção

Máquinas macOS custam mais minutos de CI: rode o *job* iOS só na *main*.

Builds reprodutíveis e cadeia de suprimentos

Build reprodutível

O mesmo código produz o mesmo artefato em qualquer máquina e momento. Exige versões fixas de tudo o que entra no *build*.

Como chegar lá

- Arquivos de trava no Git: `pubspec.lock`, catálogo de versões e travas do Gradle.
- Versão fixa do SDK no CI (Flutter, JDK).
- *Actions* fixadas por versão ou pelo *hash* do *commit*.

Ataque à cadeia de suprimentos

Comprometer uma dependência (biblioteca, *action*, imagem Docker) para atingir todos que a usam. É o risco M2 do OWASP Mobile Top 10.

Dica

Dependabot ou Renovate abrem PRs de atualização com o CI rodando: atualizar vira rotina, não evento.

Feature flags e lançamentos seguros

Feature flag

Chave remota que liga ou desliga uma funcionalidade sem publicar nova versão. Separa **implantar** o código de **liberar** a funcionalidade.

Usos

Lançar para 10 % dos usuários; desligar uma função com defeito (*kill switch*); testes A/B; liberar só para a equipe.

Atenção

Flags velhas são dívida técnica: remova-as quando a funcionalidade estiver estável.

Dart + Remote Config (Módulo 12)

```dart
final assistenteIa = rc.getBool('assistente_ia');
if (assistenteIa) {
  botoes.add(BotaoExplicarAlerta(alerta));
}
```

Kotlin

```kotlin
if (remoteConfig.getBoolean("assistente_ia")) {
  BotaoExplicarAlerta(alerta)
}
```

Desenvolvimento baseado em tronco

Todos integram na *main* com frequência, em *branches* curtas; o que não está pronto fica atrás de uma *flag*.

Decisões e trabalho em equipe

Registros de decisão de arquitetura (ADR)

docs/adr/0004-backend-supabase.md

```
# 4. Usar Supabase como backend das reservas
Data: 2026-10-20   Status: aceita

## Contexto
Reservas e relatórios são relacionais; a instituição
quer poder hospedar o backend no futuro.

## Decisão
Supabase (PostgreSQL + RLS); FCM só para push.

## Consequências
+ restrições no banco impedem reservas sobrepostas
+ migração para servidor próprio é viável
- offline exige banco local no app (Módulo 10)
```

ADR

Documento curto que registra uma decisão importante: contexto, decisão e consequências. Fica no repositório, numerado, e nunca é apagado; se mudar, um ADR novo o substitui.

Por que registrar

Daqui a um ano, ninguém lembrará por que a escolha foi feita. O ADR evita refazer discussões e ajuda quem chega ao time.

Dica

Complete com `CODEOWNERS` (quem revisa cada pasta) e um guia de contribuição.

Fechamento

Resumo

Resumo

- Dívida técnica cobra juros; registre-a e pague onde o código muda mais.
- Módulos com dependências só para baixo aceleram o *build* e organizam a equipe.
- Desempenho se mede em aparelho real e em *release*: orçamento de quadro, inicialização, memória.
- Observabilidade une *logs*, métricas e *traces*; SLOs e orçamento de erro guiam prioridades.
- CI/CD automatiza qualidade e entrega; *builds* reprodutíveis protegem a cadeia de suprimentos.
- *Feature flags* separam implantar de liberar; ADRs guardam as decisões.

Glossário (1/2)

Termo	Significado
Dívida técnica	custo futuro gerado por atalhos no código
Módulo	parte do código com fronteira e dependências explícitas
Orçamento de quadro	tempo máximo para produzir um quadro (16,7 ms a 60 Hz)
Jank	engasgo visível causado por quadros atrasados
Inicialização a frio	abertura do app sem processo existente
Vazamento de memória	objeto inútil que continua alcançável e não é liberado
Profiler	ferramenta que mede onde o tempo e a memória são gastos
Baseline Profile	lista de código pré-compilado para acelerar o app Android

Glossário (2/2)

Termo	Significado
Observabilidade	entender o sistema a partir de *logs*, métricas e *traces*
ID de correlação	identificador que acompanha uma operação do app ao servidor
SLI / SLO	indicador de serviço / meta para esse indicador
Orçamento de erro	margem de falhas permitida pelo SLO
CI / CD	integração contínua / entrega ou implantação contínua
***Build* reprodutível**	mesmo código gera o mesmo artefato em qualquer máquina
Feature flag	chave remota que liga ou desliga funcionalidades
ADR	registro curto de uma decisão de arquitetura

Questões de revisão

1. Dê um exemplo de dívida técnica prudente e deliberada e diga quando ela deve ser paga.
2. Por que *features* não devem depender umas das outras?
3. Quanto tempo há para cada quadro a 120 Hz? O que acontece se ele for excedido?
4. Por que medir desempenho em modo *debug* leva a conclusões erradas?
5. Cite duas causas de vazamento de memória e como evitá-las nas duas trilhas.
6. Com SLO de 99 % sem falhas e 50 000 sessões no mês, qual é o orçamento de erro?
7. Qual a diferença entre entrega contínua e implantação contínua?
8. Como uma *feature flag* reduz o risco de um lançamento?

Exercícios

Exercício 1 — Modularização

Divida o LabMonitor em `core` e `feature` (Gradle ou *pub workspace*) e desenhe o grafo de dependências.

Exercício 2 — Desempenho

Meça a inicialização a frio e a rolagem do painel em *release* num aparelho modesto, aplique uma otimização e meça de novo.

Exercício 3 — Observabilidade

Defina dois SLIs e um SLO para o LabMonitor e implemente o ID de correlação entre app e API.

LabMonitor — Etapa 18

Pipeline completo no GitHub Actions (qualidade em todo PR, artefato assinado na *main*), uma *feature flag* para o assistente de IA e dois ADRs registrados.

Para aprofundar

- Modularização no Android: `https://developer.android.com/topic/modularization`
- Desempenho no Android: `https://developer.android.com/topic/performance`
- Desempenho no Flutter: `https://docs.flutter.dev/perf`
- *Pub workspaces*: `https://dart.dev/tools/pub/workspaces`
- Livros de SRE do Google (gratuitos): `https://sre.google/books`
- GitHub Actions: `https://docs.github.com/actions`

Respostas das questões de revisão

1. Lançar a reserva sem o filtro avançado para cumprir a data da feira, sabendo que o código da lista será refeito. Deve ser paga logo após o lançamento, antes que outras funções se apoiem nele.
2. Para evitar ciclos e permitir que cada *feature* seja compilada, testada e mantida isoladamente; o que elas compartilham vai para um módulo `core`.
3. 8,3 ms. O quadro é descartado e a tela repete o anterior: o usuário vê um engasgo.
4. Em *debug* o código roda com verificações extras e sem otimizações (JIT no Flutter); os gargalos e os tempos são diferentes dos reais.
5. Coletar `Flow` fora do ciclo de vida (usar `repeatOnLifecycle`) e não descartar *controllers* ou assinaturas (`dispose` e `cancel`).
6. 1 % de 50 000 = 500 sessões com falha no mês.
7. Na entrega contínua, cada versão fica pronta para publicar, mas alguém decide quando; na implantação contínua, ela é publicada automaticamente.
8. Permite liberar para poucos usuários, observar e desligar na hora se algo der errado, sem esperar uma nova versão passar pela loja.

Fim do Módulo 18

Próximo: Módulo 19 — Multiplataforma (trilha Kotlin) ou código nativo, web, desktop e embarcados (trilha Flutter).